lec513

Dylan Tang

May 2026

1 Review of GD and Backpropagation

1.0.1 Gradient Descent

Gradient descent is given by:

$$W^{(t+1)} = W^{(t)} - \eta \nabla_W \mathcal{L}\left(W^{(t)}\right)$$

1.0.2 Backward Propogation

- For a hidden layer ℓ, recall that

$$Z^{(\ell)} = W^{(\ell)} A^{(\ell-1)} + b^{(\ell)}, \qquad A^{(\ell)} = \sigma\left(Z^{(\ell)}\right).$$

Using the chain rule, the gradient with respect to $W^{(\ell)}$ is

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial A^{(\ell)}} \frac{\partial A^{(\ell)}}{\partial Z^{(\ell)}} \frac{\partial Z^{(\ell)}}{\partial W^{(\ell)}}.$$

Then,

$$\frac{\partial A^{(\ell)}}{\partial Z^{(\ell)}} = \sigma'(Z^{(\ell)})$$

and

$$\frac{\partial Z^{(\ell)}}{\partial W^{(\ell)}} = (A^{(\ell-1)})^T$$

which gives

$$\boxed{\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \left[\frac{\partial \mathcal{L}}{\partial A^{(\ell)}} \odot \sigma'\left(Z^{(\ell)}\right)\right]\left(A^{(\ell-1)}\right)^T}$$

Note that

$$\mathcal{L} = f(Y, \hat{Y}) = f(Y, \sigma(W^{(L)} A^{(L-1)}) + b^{(L)}) = f(Y, \sigma(W^{(L)} \sigma(W^{(L-1)} A^{(L-2)}) + b^{(L-1)}) + b^{(L)}) = \dots$$

2 Convolution

$$X = [a, b, c, d, e], \qquad Y = [-1, 1]$$

- Think of X as the input signal.
- Think of Y as the filter/kernel.
- We slide Y across X, computing the dot product at each slide.

X	a	b	c	d	e
Y	-1	1			

$$(-1)a + (1)b = -a + b$$

X	a	b	c	d	e
Y		-1	1		

$$(-1)b + (1)c = -b + c$$

X	a	b	c	d	e
Y			-1	1	

$$(-1)c + (1)d = -c + d$$

X	a	b	c	d	e
Y				-1	1

$$(-1)d + (1)e = -d + e$$

$$X * Y = [-a + b,\ -b + c,\ -c + d,\ -d + e]$$

- Convolution length is $n - k + 1$, where $len(X) = 5$, $len(Y) = 2$
- This filter computes local differences between neighboring entries.

Consider the following image:

$$X = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

and had the filter

$$Y = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

Then,

$$X * Y = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$$

For the example above,

- 1 means the filter found a matching pattern
- -1 means the filter found an opposite pattern
- 0 means the filter did not match in either direction

A CNN learns the combination of filters at each hidden layer that results in the best prediction.

Consider

$$X = \begin{bmatrix} a & b & c & d & e \end{bmatrix}$$

Then, let

$$m_1 = \begin{bmatrix} m_{11} & m_{12} \end{bmatrix}$$

and

$$m_2 = \begin{bmatrix} m_{21} & m_{22} \end{bmatrix}$$

Then,

$$X * m_1 * m_2 = \begin{bmatrix} (am_{11} + bm_{12})m_{21} + (bm_{11} + cm_{12})m_{22} \\ (bm_{11} + cm_{12})m_{21} + (cm_{11} + dm_{12})m_{22} \\ (cm_{11} + dm_{12})m_{21} + (dm_{11} + em_{12})m_{22} \end{bmatrix}.$$

- Each element in the final series of convolutions contains elements in $X, m_1, m_2 \implies$ convolution gives each element information about the entire network
- So, convolution is a very fast way to diseeminate information to all nodes of a layer
- Cascades of convolutions are called a **receptive field**. The end result contains information of all parts of the cascade.